Databases and its Applications

Relational Modelling

Pooja T S

Computer Applications

Databases and its Applications

Relational Modelling

SQL: Nested Queries, Subqueries

Pooja T S

Computer Applications

Databases and its Applications

Introduction to Subqueries

- A **subquery** (or nested query) is a query written inside another SQL statement.
- The inner query runs first, and its result is used by the outer query.
- Subqueries are enclosed within parentheses `( )`.
- PostgreSQL allows subqueries in `SELECT`, `FROM`, and `WHERE` clauses.

Databases and its Applications

Why Use Subqueries?

PES UNIVERSITY

- Subqueries make SQL more modular and expressive.
- Common use cases include:
  - Filtering results using another query.
  - Comparing aggregated data to individual rows.
  - Creating derived datasets on the fly.
  - Replacing temporary tables.

Types of Subqueries

- PostgreSQL supports:
  - **Single-row subquery** – returns exactly one value.
  - **Multi-row subquery** – returns multiple values.
  - **Correlated subquery** – executed once per row of outer query.
  - **Subquery in FROM clause** – acts like a temporary table.

Databases and its Applications

Single-row Subqueries – Concept

- Single-row subqueries return one value that can be used with comparison operators.
- Usually appear with =, >, <, >=, or <=.
- Example: finding students whose CGPA equals the maximum CGPA in the table.

Databases and its Applications

Single-row Subquery – Example

- **Command:**

```
SELECT sname, cgpa
FROM student
WHERE cgpa = (SELECT MAX(cgpa) FROM student);
```

- **Output (sample):**

```
 sname | cgpa
-------+------
 Kavya | 9.10
(1 row)
```

- The inner query returns the highest CGPA; the outer query finds the student(s) who match it.

Databases and its Applications

Multi-row Subqueries – Concept

- Multi-row subqueries return multiple values.
- They are used with operators like `IN`, `ANY`, or `ALL`.
- Example: selecting students whose department is among the top-performing ones.

Databases and its Applications

Using IN with Subquery

- **Command:**

```
SELECT sname, deptid
FROM student
WHERE deptid IN (SELECT deptid FROM department
WHERE deptname IN ('CSE','BCA'));
```

- **Output (sample):**

```
 sname | deptid
-------+--------
 Ravi  | 1
 Anita | 3
(2 rows)
```

Databases and its Applications

Using ANY and ALL

PES UNIVERSITY

- **ANY** – true if the condition is true for **at least one** value.
- **ALL** – true if the condition is true for **all** values.
- Examples:
  - `WHERE cgpa > ANY (SELECT cgpa FROM student WHERE deptid = 1);` → students with CGPA greater than **any one** in dept 1.
  - `WHERE cgpa > ALL (SELECT cgpa FROM student WHERE deptid = 1);` → students with CGPA greater than **all** in dept 1.

Databases and its Applications

Correlated Subqueries – Concept

- A **correlated subquery** references columns from the outer query.
- It executes once for each row processed by the outer query.
- Common for comparisons like “find students whose CGPA is above their department average.”
- **Command:**

```
SELECT sname, cgpa, deptid
FROM student s
WHERE cgpa > (SELECT AVG(cgpa)
     FROM student
     WHERE deptid = s.deptid);
```

Correlated Subquery – Example

- **Output (sample):**

```
 sname | cgpa | deptid
--------+------+--------
 Ravi | 8.40 | 1
 Kavya | 9.10 | 2
(2 rows)
```

- Each department's average is computed and compared against individual students.

Databases and its Applications

Subquery in FROM Clause

- Subqueries can appear in the `FROM` clause and act like a temporary derived table.
- Must have an alias.
- Useful for simplifying multi-step analysis.

Databases and its Applications

FROM Clause Subquery – Example

- **Command:**

```
SELECT deptid, avg_cgpa
FROM (SELECT deptid, AVG(cgpa) AS avg_cgpa
    FROM student GROUP BY deptid) AS dept_avg
WHERE avg_cgpa > 8.0;
```

- **Output (sample):**

```
 deptid | avg_cgpa
--------+----------
 1 | 8.40
 2 | 9.10
(2 rows)
```

Databases and its Applications

Subquery in SELECT Clause

- Subqueries can be used as computed columns in `SELECT`.
- Common for adding derived values like averages or totals.

SELECT Subquery – Example

- **Command:**

```
SELECT sname, cgpa,
    (SELECT deptname FROM department d WHERE
d.deptid = s.deptid) AS department
FROM student s;
```

- **Output (sample):**

```
 sname | cgpa | department
-------+------+------------
 Ravi | 8.40 | CSE
 Kavya | 9.10 | ECE
 Anita | 7.80 | BCA
(3 rows)
```

Databases and its Applications

Using EXISTS with Subqueries

PES UNIVERSITY

- The `EXISTS` operator tests if a subquery returns any rows.
- It returns TRUE if the subquery produces at least one result.
- Common for “existence check” queries.
- **Command:**

```
SELECT deptname
FROM department d
WHERE EXISTS (SELECT 1 FROM student s WHERE
s.deptid = d.deptid);
```

EXISTS – Example

- **Output (sample):**

```
 deptname
-----------
 CSE
 ECE
 BCA
(3 rows)
```

- Departments without students are automatically excluded.

Databases and its Applications

Subquery Placement – Summary

- **In WHERE:** filters main query results using a subquery.
- **In FROM:** provides a derived temporary table.
- **In SELECT:** returns computed or related values per row.
- **With EXISTS / IN:** checks for matching existence conditions.

Databases and its Applications

Performance Notes on Subqueries

- PostgreSQL optimizes subqueries by converting them to joins where possible.
- Correlated subqueries are slower for large datasets.
- Always test complex subqueries with `EXPLAIN ANALYZE`.
- Use `WITH` (Common Table Expressions) for clarity in multi-level nesting.

Databases and its Applications

Best Practices for Subqueries

- Prefer joins when working with large tables.
- Use subqueries for clarity when the logic is nested or hierarchical.
- Always alias subqueries in the `FROM` clause.
- For performance and readability, refactor multi-level nesting into CTEs (`WITH`).

Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 233